

Hafta 2: Programlama Temelleri

Bilgisayarla Konuşma Sanatı

Programlama, modern dünyanın dilini öğrenmek demektir. Bu hafta, bilgisayarların nasıl düşündüğünü ve onlarla nasıl etkili bir şekilde iletişim kurabileceğimizi keşfedeceğiz.

Haftanın Yol Haritası

Bu hafta boyunca programlamanın temel yapı taşlarını adım adım öğreneceğiz. Her bir kavram, bir sonraki için sağlam bir temel oluşturacak.

01

Programlama Nedir?

Bilgisayarla konuşmanın temellerini ve programlamanın önemini anlayacağız

02

Algoritmik Düşünme

Problemleri adım adım nasıl çözeceğimizi öğreneceğiz

03

Değişkenler

Bilgiyi bellekte nasıl saklayıp kullanacağımızı keşfedeceğiz

04

Koşullar (if/else)

Bilgisayara nasıl karar verdireceğimizi öğreneceğiz

05

Döngüler (for/while)

Tekrarlayan işleri nasıl otomatize edeceğimizi göreceğiz

06

Neden Python?

İlk dilimiz olarak Python'u neden seçtiğimizi anlayacağız

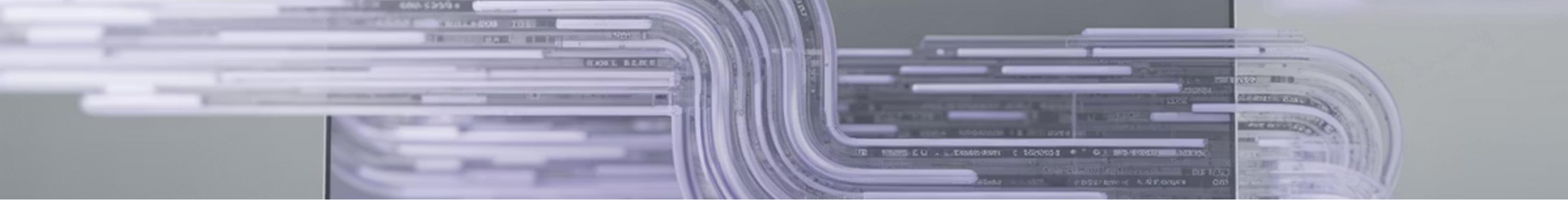
Bilgisayarla Konuşma Sanatı

Programlama, bir bilgisayara ne yapması gerektiğini anladığı bir dilde (programlama dili) adım adım, son derece net bir şekilde anlatma işlemidir. Bilgisayarlar son derece güçlü makinelerdir, ancak kendi başlarına düşünemezler veya karar veremezler.

Önemli Analoji: Bilgisayarı, çok hızlı ama kendi başına düşünemeyen bir asistan gibi düşünün. Ona ne yapacağını en ince ayrıntısına kadar, hatasız ve belirsizlik içermeyecek şekilde tarif etmelisiniz.

1. **Tarif (Algoritma)**
   Adım adım talimatlar
2. **Malzemeler (Veri)**
   İşlenecek bilgi
3. **Aşçı (İşlemci)**
   Talimatları çalıştıran
4. **Yemek (Çıktı)**
   Sonuç





Neden Python?

Öğrenmesi Kolay, Gücü Yüksek

Python, günümüzün en popüler ve çok yönlü programlama dillerinden biridir. Hem yeni başlayanlar hem de profesyoneller tarafından tercih edilmektedir.

Okunabilir Sözdizimi

Neredeyse İngilizce gibi yazılır ve okunur. Bu özellik, öğrenme sürecini hızlandırır ve kodun anlaşılmasını kolaylaştırır.

Çok Yönlü Kullanım

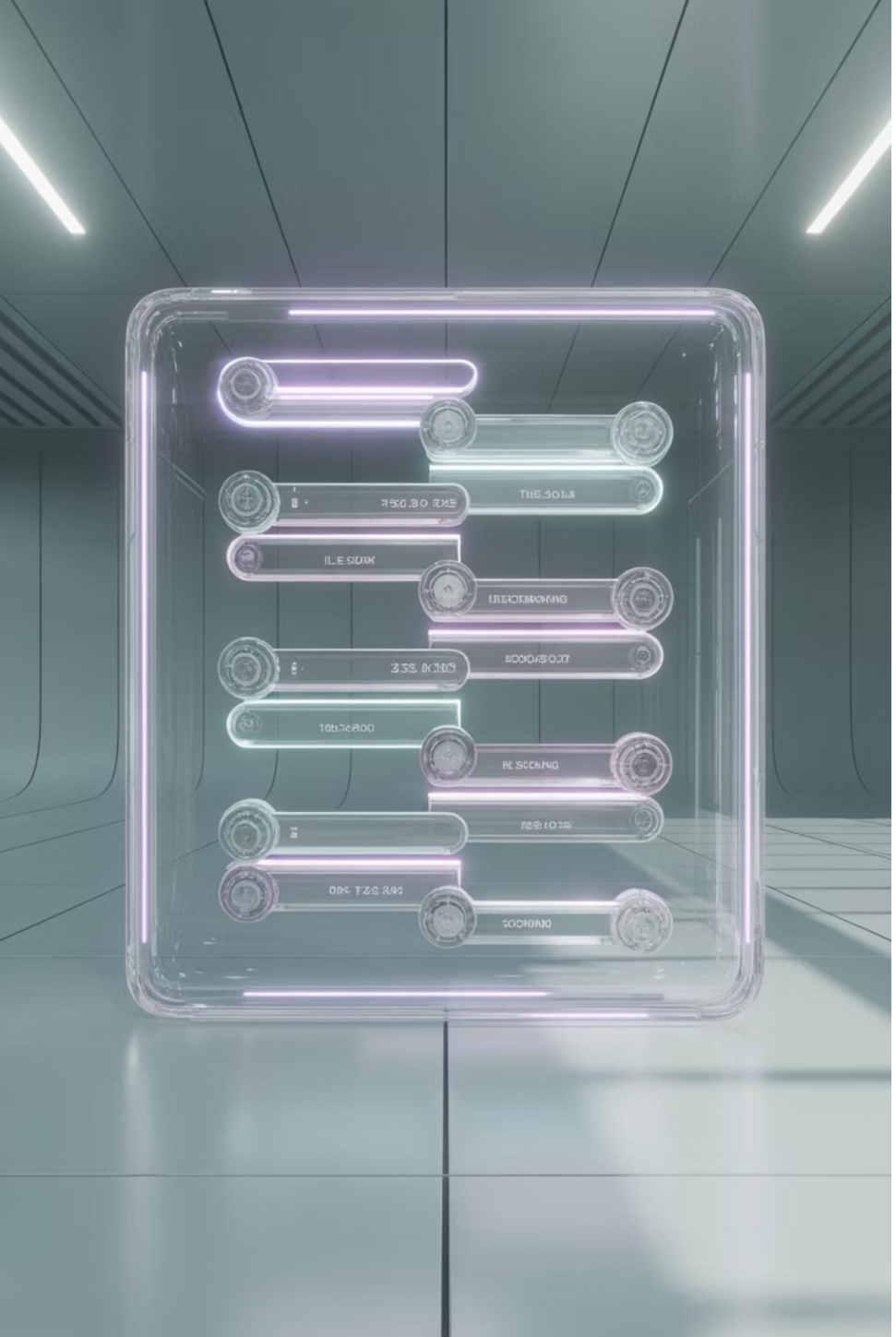
Web geliştirmeden yapay zekâya, veri biliminden oyun programlamaya kadar neredeyse her alanda kullanılabilir.

Zengin Kütüphane Ekosistemi

Başkalarının yazdığı binlerce hazır kütüphane, işlerinizi inanılmaz derecede hızlandırır ve karmaşık problemleri kolayca çözmenizi sağlar.

Geniş ve Destekleyici Topluluk

Milyonlarca Python geliştiricisi var. Takıldığınız her konuda internette kolayca yardım, tutorial ve çözüm bulabilirsiniz.



Bölüm 1: Algoritmik Düşünme

Her Şey Bir Tarif ile Başlar

Algoritma, belirli bir problemi çözmek veya bir görevi tamamlamak için izlenen, sonlu ve net adımlar dizisidir. Programlamanın en temel yapı taşıdır ve iyi bir programcı olmak için algoritmik düşünme becerisini geliştirmek şarttır.

Başlangıç ve Son Var

Her algoritmanın net bir başlangıç noktası ve bitiş noktası bulunur

Adımlar Net ve Belirli

Hiçbir adım belirsizlik içermez, her şey açık ve anlaşılırdır

Tekrarlanabilir Sonuç

Herkes tarafından uygulandığında tutarlı ve aynı sonucu verir

Gerçek Hayattan Algoritma Örneği

Günlük Hayattaki Algoritmalar: Çay Demleme



Aslında her gün onlarca algoritmayı bilinçsizce uyguluyoruz. İşte çok tanıdık bir örnek:

BAŞLA - Çaydanlığa su koy

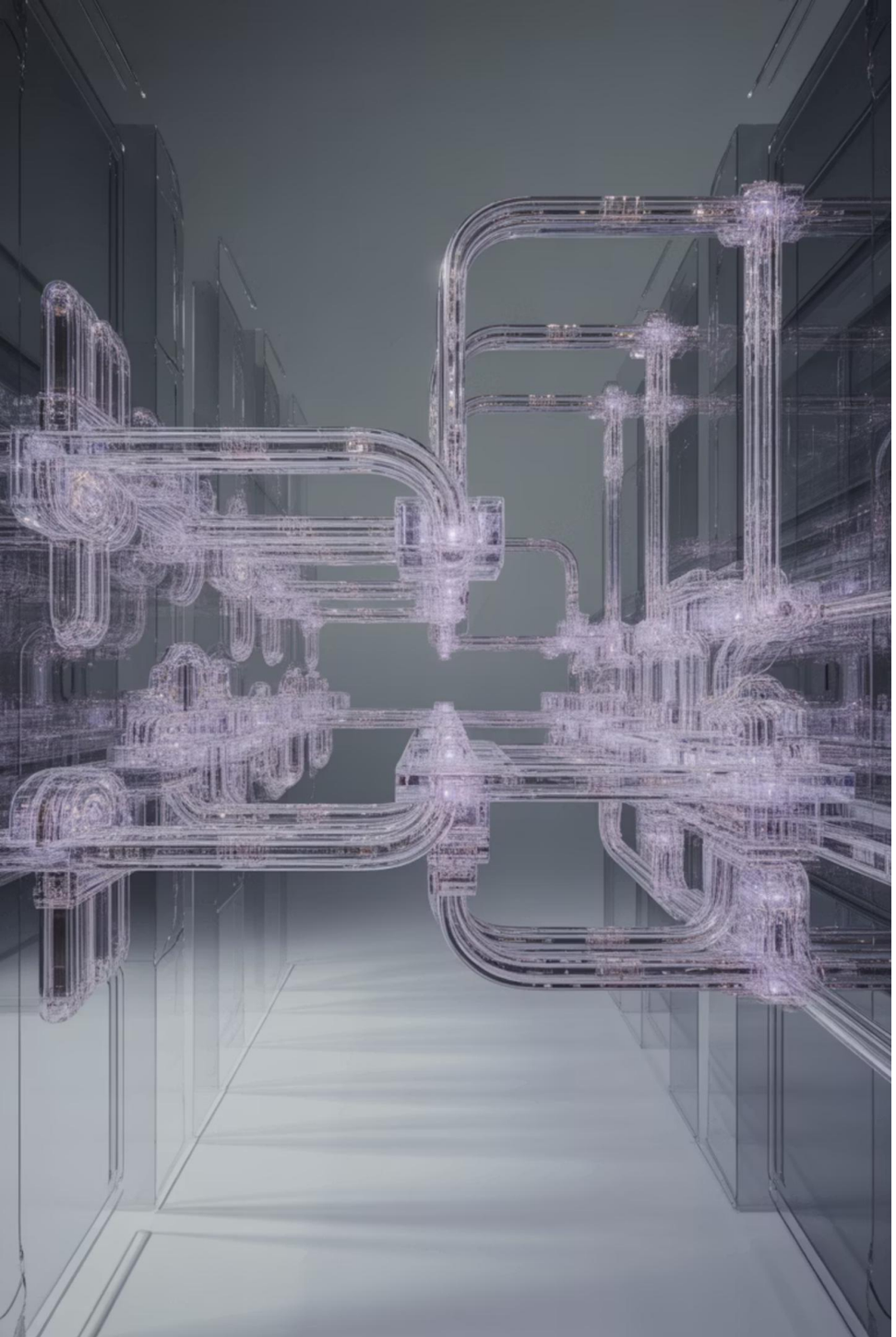
Ocağı yak ve suyu kaynat

Karar Noktası: Su kaynadı mı?

Demliğe çay koy, kaynamış suyu dök

15 dakika bekle - **BİTİR**

Düşünün: Bu adımlardan birini atlarsak veya sırasını değiştirirsek ne olur? Belki de yakmadan suyu kaynatmaya çalışırız veya çay koymadan su dökeriz. Bilgisayar da tam olarak bu yüzden net ve *doğru sırayla verilmiş* adımlara ihtiyaç duyar!



Akış Şemaları

Fikirleri Şekle Dökmek

Akış şemaları, algoritmaları görselleştirmek için kullanılan standart şekillerdir. Karmaşık süreçleri anlaşılır hale getirir ve ekip üyeleri arasında ortak bir dil oluşturur.

Elips (Oval)

Algoritmanın başlangıç ve bitiş noktalarını temsil eder. Her akış şemasında mutlaka bulunmalıdır.

Dikdörtgen

İşlem adımlarını veya atama komutlarını gösterir. Örneğin: "Suya şeker ekle" veya "x = 5"

Baklava Dilimi

Karar verme noktalarını ifade eder. İçinde bir soru bulunur: "Su kaynadı mı?" gibi.

Paralelkenar

Veri girişi (kullanıcıdan bilgi almak) veya veri çıktısı (ekrana yazdırmak) işlemlerini gösterir.

Sözde Kod (Pseudocode)

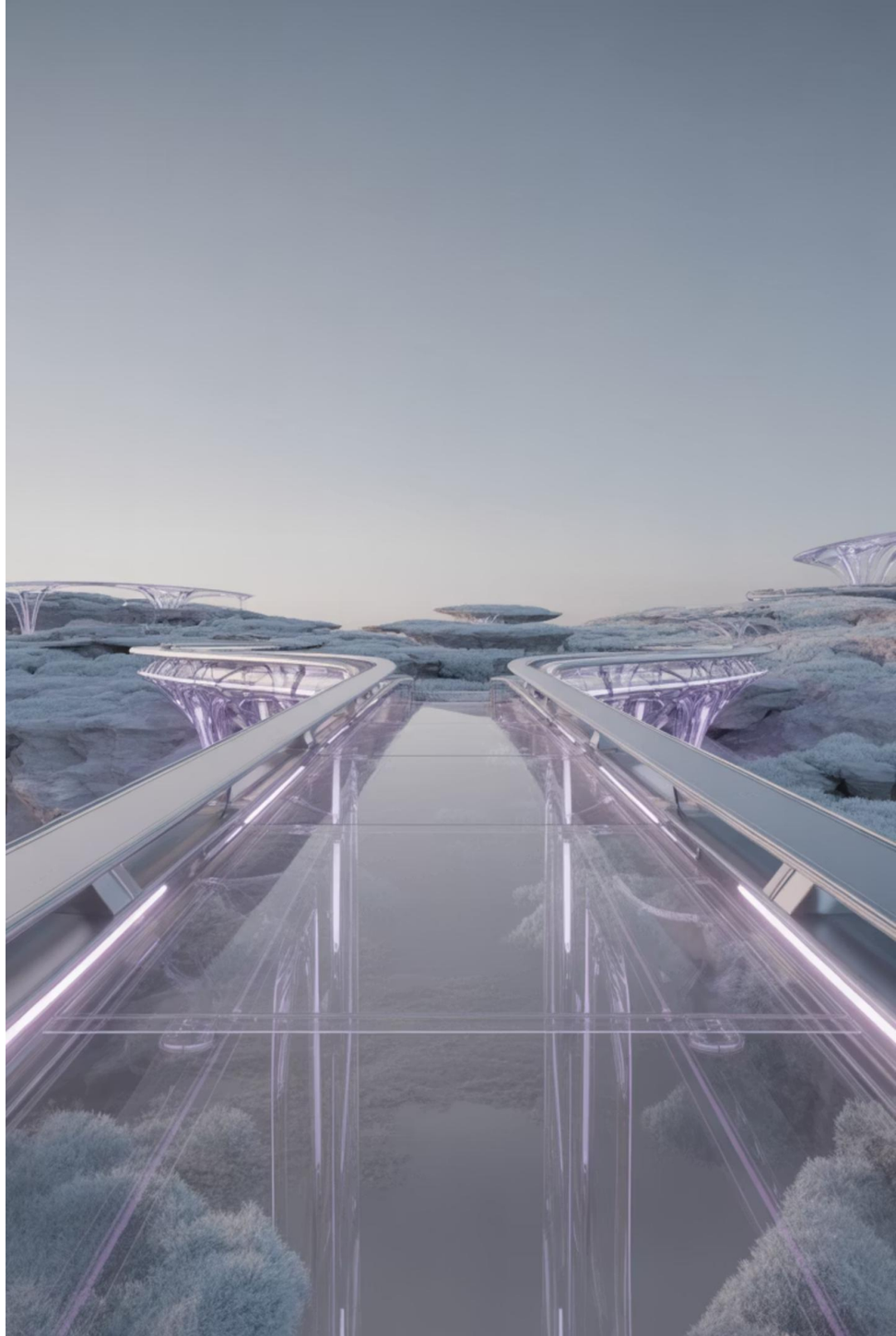
Programlamaya Geçiş Köprüsü

Sözde kod, herhangi bir programlama dilinin katı kurallarına bağlı kalmadan, algoritmayı günlük konuşma diline yakın bir şekilde yazmaktır. Fikrinizi hızlıca kâğıda dökmenizi ve mantık hatalarını erken tespit etmenizi sağlar.

Sözde kod yazarken **dilin kurallarından çok, mantığın doğruluğu** önemlidir. Bu yöntem, kodu yazmadan önce düşünmenize ve planlama yapmanıza yardımcı olur.

Örnek: Sınav Notu Hesaplama

```
BAŞLA OKU vize_notu OKU final_notu
ortalama = (vize_notu * 0.4) + (final_notu *
(final_notu * 0.6)  EĞER ortalama >= 50
YAZDIR "Geçtiniz! Ortalamanız: " + ortalama
ortalama DEĞİLSE YAZDIR "Kaldınız.
Ortalamanız: " + ortalama BİTİR
```





Bölüm 2: Değişkenler ve Veri Tipleri

Programın Hafızası

Değişkenler, program çalışırken verileri geçici olarak saklamak için kullanılan isimlendirilmiş hafıza alanlarıdır. Her program mutlaka verilerle çalışır: kullanıcı adı, yaş, hesap bakiyesi, oyun skoru gibi. İşte bu verileri tutmak için değişkenlere ihtiyacımız vardır.

ad = "Ahmet"

Metin bilgisi içeren bir kutu

yas = 20

Sayısal bilgi içeren bir kutu

ortalama = 85.5

Ondalıklı sayı içeren bir kutu

Değişkenlerin güzel yanı, içerdikleri değerleri **istediğiniz zaman değiştirebilmenizdir**. Örneğin, oyun skorunuz arttıkça "skor" değişkeninin değeri de güncellenir.

Temel Veri Tipleri

Bilginin Farklı Türleri

Python'da her değişken bir veri tipine sahiptir. Veri tipi, o değişkenin ne tür bilgi sakladığını ve onunla ne tür işlemler yapılabileceğini belirler.

Integer (Tam Sayı)

Ondalık kısım olmayan sayılar. Örnek: `yas = 21`, `ogrenci_sayisi = 150`

Kullanım: Yaş, adet sayma, sıralama gibi tam sayı gerektiren durumlar.

Float (Ondalıklı Sayı)

Kesirli kısım içeren sayılar. Örnek: `pi = 3.14159`, `not_ortalamasi = 85.5`

Kullanım: Ortalama, ölçüm, para birimi gibi hassas hesaplamalar.

String (Metin)

Harf, rakam ve sembol içeren metinler. Örnek: `isim = "Ayşe"`, `bolum = "Bilgisayar Mühendisliği"`

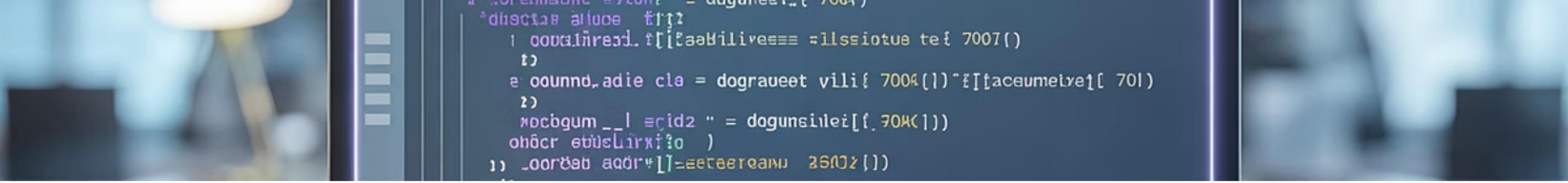
Kullanım: İsimler, adresler, mesajlar, her türlü metinsel bilgi.

Boolean (Mantıksal)

Sadece iki değer alabilir: `True` (Doğru) veya `False` (Yanlış)

Örnek: `is_active = True`, `mezun_mu = False`

Kullanım: Durum kontrolü, karar verme, koşul testleri.



Python'da Değişken Tanımlama

Pratiğe Dökme Zamanı

Python'da bir değişken oluşturmak inanılmaz derecede kolaydır. Değişken adı yazıp, eşittir işareti koyup, değeri yazmak yeterlidir. Python, değişkenin tipini atanan değere göre **otomatik olarak** anlar.

> **Önemli Not:** Python'da değişkenin tipini önceden belirtmemize gerek yoktur. Python bunu atanan değere göre kendisi halleder. Buna *"Dinamik Tipli Dil"* denir ve kodlamayı çok kolaylaştırır!

Python Kod Örneği:

```
# Metin (String)ogrenci_adi = "Mehmet Veli"# Tam Sayı (Integer)dogum_yili = 2004# Ondalıklı Sayı (Float)boy = Sayı (Float)boy = 1.78# Mantıksal (Boolean)kayit_dondurdu_mu = False# Değişkenleri kullanmaprint(ogrenci_adi)print("Yaş:", 2025 - dogum_yili)
```

Bölüm 3: Koşullu İfadeler

Karar Verme Zamanı

Koşullar, programın akışını belirli bir durumun doğru ya da yanlış olmasına göre değiştiren yapılardır. Gerçek hayatta sürekli kararlar veririz ve programlarımız da aynı şeyi yapmalıdır.

1

EĞER (if)

Hava yağmurluysa → "Şemsiyeni al"

2

DEĞİLSE EĞER (elif)

Hava güneşliyse → "Güneş gözlüğü tak"

3

DEĞİLSE (else)

Diğer durumlarda → "Hava normal"

Koşullar sayesinde programlarımız esnek ve akıllı hale gelir. Farklı durumlara farklı tepkiler verebilir, kullanıcı girdilerine göre davranışını değiştirebilir.



Karşılaştırma ve Mantıksal Operatörler

Koşulları Oluşturan Yapı Taşları

Koşulları oluştururken belirli operatörleri kullanırız. Bu operatörler, değerleri karşılaştırmamıza ve karmaşık mantıksal ifadeler oluşturmamıza olanak tanır.

Karşılaştırma Operatörleri

- `==` : Eşit mi? (Dikkat: Atama için `=`, karşılaştırma için `==`)
- `!=` : Eşit değil mi?
- `>` : Büyük mü?
- `<` : Küçük mü?
- `>=` : Büyük veya eşit mi?
- `<=` : Küçük veya eşit mi?

Örnek: `yas >= 18` ifadesi, yaş 18 veya daha büyükse `True` döner.

Mantıksal Operatörler

- `and` : Her iki koşul da doğru olmalı
- `or` : Koşullardan en az biri doğru olmalı
- `not` : Koşulun tersini alır (True → False, False → True)

Örnek: `yas >= 18 and ehliyet_var == True` ifadesi, hem yaş uygunsa hem de ehliyet varsa `True` döner.

Python'da if-elif-else Örneği

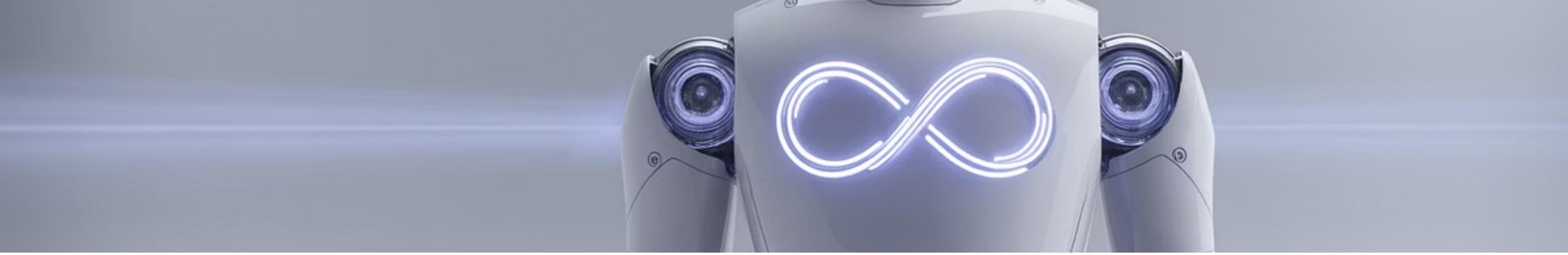
Kod ile Karar Vermek

Hava Durumu Uygulaması

```
hava_durumu = "parçalı bulutlu"if hava_durumu == "yağmurlu":    print("Yanına şemsiye almayı unutma!")    print("Ayakkabıların
print("Ayakkabıların ıslanabilir.")    elif hava_durumu == "güneşli":    print("Güneş kremi sürmek için iyi bir gün.")    print("Şapka
print("Şapka takmayı düşün.")    else:    print("Hava ne sıcak ne soğuk, tam tadında.")    print("Rahat bir gün geçireceksin!")
geçireceksin!")
```



Kritik Önemli: Python'da girintileme (indentation) sadece görsel bir tercih değildir,



Bölüm 4: Döngüler

Yorulmayan Çalışanlar

Döngüler, bir kod bloğunu belirli bir koşul sağlandığı sürece veya belirli sayıda **tekrar tekrar çalıştırmak** için kullanılır. Programlamanın en güçlü özelliklerinden biridir.

Neden Gerekli?

"Merhaba Dünya" yazısını 1000 kere ekrana yazdırmak istiyorsunuz. Tek tek mi yazarsınız, yoksa bilgisayara "bunu 1000 kere yap" mı dersiniz?

Zaman Tasarrufu

İnsanların saatler süren tekrarlayan işlerini bilgisayarlar saniyeler içinde halledebilir.

Hata Azaltma

Tekrarlayan işlerde insan hatası kaçınılmazdır. Döngüler tutarlı ve hatasız çalışır.

for Döngüsü

Belirli Sayıda Tekrar

`for` döngüsü, genellikle bir listenin veya belirli bir aralığın üzerinde dolaşmak için kullanılır. **Tekrar sayısı önceden belli olduğunda** tercih edilir.



Örnek: 1'den 5'e Kadar Sayıları Yazdırma

```
# range(1, 6) fonksiyonu# 1, 2, 3, 4, 5 sayılarını üretir# (6 dahil değildir!)for sayi in range(1, 6): range(1, 6):    print("Sıradaki sayı:", sayi)    print("Karesini alırsak:", sayi * sayi)    print("Döngü print("Döngü bitti!")
```

`range()` fonksiyonu çok kullanışlıdır:

- `range(5)` → 0'dan 4'e kadar
- `range(1, 10)` → 1'den 9'a kadar
- `range(0, 20, 2)` → 0'dan 20'ye, 2'şer 2'şer

while Döngüsü

Koşul Sağlandığı Sürece Tekrar

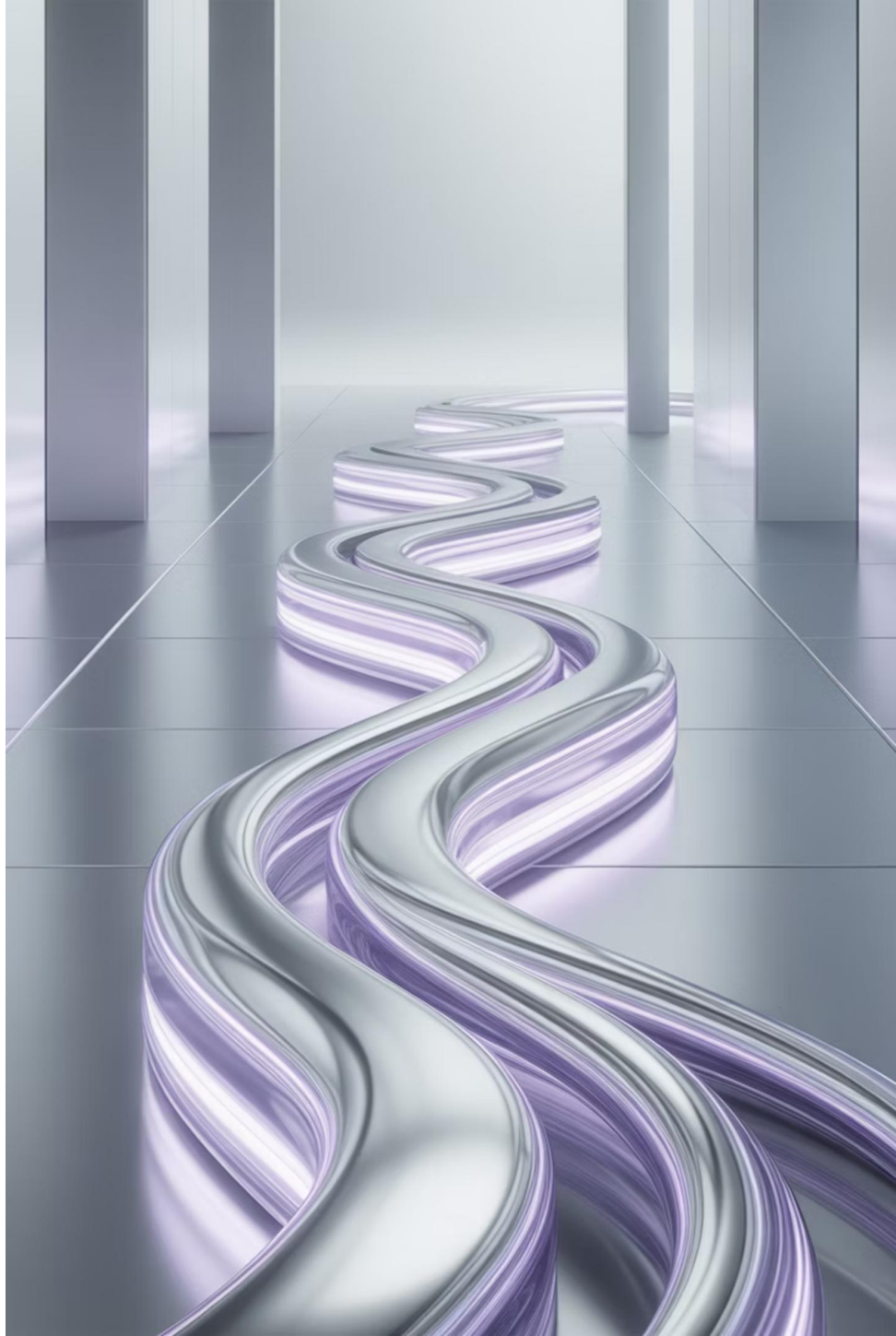
`while` döngüsü, içindeki koşul `True` olduğu sürece çalışmaya devam eder. **Ne kadar tekrar edeceğini önceden bilemediğimiz** durumlarda kullanışlıdır.

Basit Sayaç Örneği

```
sayac = 1while sayac <= 5:    print("Sıradaki print("Sıradaki sayı:", sayac)    sayac = sayac + 1 sayac + 1    # Bu satır kritik!    # Olmazsa sonsuz sonsuz döngü!    print("Döngü tamamlandı!") tamamlandı!")
```

Kullanıcı Girişi İle Örnek

```
dogru_sifre = "12345"girilen_sifre = ""while ""while girilen_sifre != dogru_sifre: girilen_sifre girilen_sifre = input("Şifre girin: ")  if girilen_sifre != dogru_sifre: print("Yanlış şifre, şifre, tekrar dene!") print("Giriş başarılı!")
```





Döngülerde Dikkat Edilmesi Gerekenler

Sonsuz Döngü Tehlikesi!

`while` döngülerinde en büyük tehlike, sonsuz döngüye girmektir. Döngüyü sonlandıracak koşulun bir noktada mutlaka `False` olmasını sağlamalısınız.

✓ Doğru Kullanım

```
sayac = 0while sayac < 5:    print(sayac)    sayac = sayac + 1  # Koşul değişiyor!
```

✗ Yanlış Kullanım (Sonsuz Döngü!)

```
sayac = 0while sayac < 5:    print(sayac)    # sayac hiç artmıyor!    # Bu kod sonsuza kadar çalışır!
```

Acil Durum İpucu: Eğer programınız dondu ve durmuyor gibi görünüyorsa, muhtemelen sonsuz döngüye girmiştir. Çoğu editörde `Ctrl+C` tuşlarına basarak programı durdurabilirsiniz.

Pratik Örnek: Basit Sayı Tahmin Oyunu

Öğrendiklerimizi Birleştirelim

Şimdi bugün öğrendiğimiz tüm kavramları bir araya getirerek eğlenceli bir oyun tasarlayalım. Bu örnekte değişkenler, koşullar ve döngüler bir arada kullanılıyor.

Sözde Kod (Algoritma)

```
BAŞLA  rastgele_sayi = 1 ile 100 arasında seç  tahmin_hakki = 5    DÖNGÜ (BİLİNCEYE veya HAK BİTENE KADAR):    Kullanıcıdan tahmin AL      EĞER tahmin < rastgele_sayi:     YAZDIR "Daha büyük bir sayı dene."         DEĞİLSE EĞER tahmin > rastgele_sayi:     YAZDIR "Daha küçük bir sayı dene."         DEĞİLSE:     YAZDIR "Tebrikler, bildiniz!"     Döngüden ÇIK         tahmin_hakki'ni 1 azalt     DÖNGÜ BİTİŞ    EĞER tahmin_hakki == 0:    YAZDIR "Maalesef hakkınız bitti."    YAZDIR "Doğru sayı: " + rastgele_sayi    BİTİR
```

Oyunun Özellikleri

- **Değişkenler:** Rastgele sayı, tahmin, kalan hak
- **Döngü:** Kullanıcı bilene veya hak bitene kadar devam eder
- **Koşullar:** Tahmini değerlendirir ve ipucu verir
- **Karşılaştırma:** Büyük/küçük kontrolleri yapar

Bonus Görev: Bu algoritmanın Python kodunu yazabilir misiniz? İpucu: `import random` modülünü kullanarak `random.randint(1, 100)` ile rastgele sayı üretebilirsiniz.

Modern Programlama Pratikleri

Daha Temiz, Daha Organize ve Daha Güçlü Kod Yazmak



Haftanın Yol Haritası

Bu hafta, profesyonel yazılım geliştirmenin temel taşlarını keşfedeceğiz. Kodunuzu nasıl daha modüler, yönetilebilir ve ekip çalışmasına uygun hale getirebileceğinizi öğreneceksiniz.

01

Fonksiyonlar

Kodun yapı taşlarını anlayalım

02

Modüler Programlama

Kodumuzu nasıl organize ederiz?

03

Versiyon Kontrolü

Neden her mühendisin bilmesi gerekir?

04

Git & GitHub

Versiyon kontrolünün süper kahramanları

05

Ekip Çalışması

Birlikte nasıl kod yazarız?

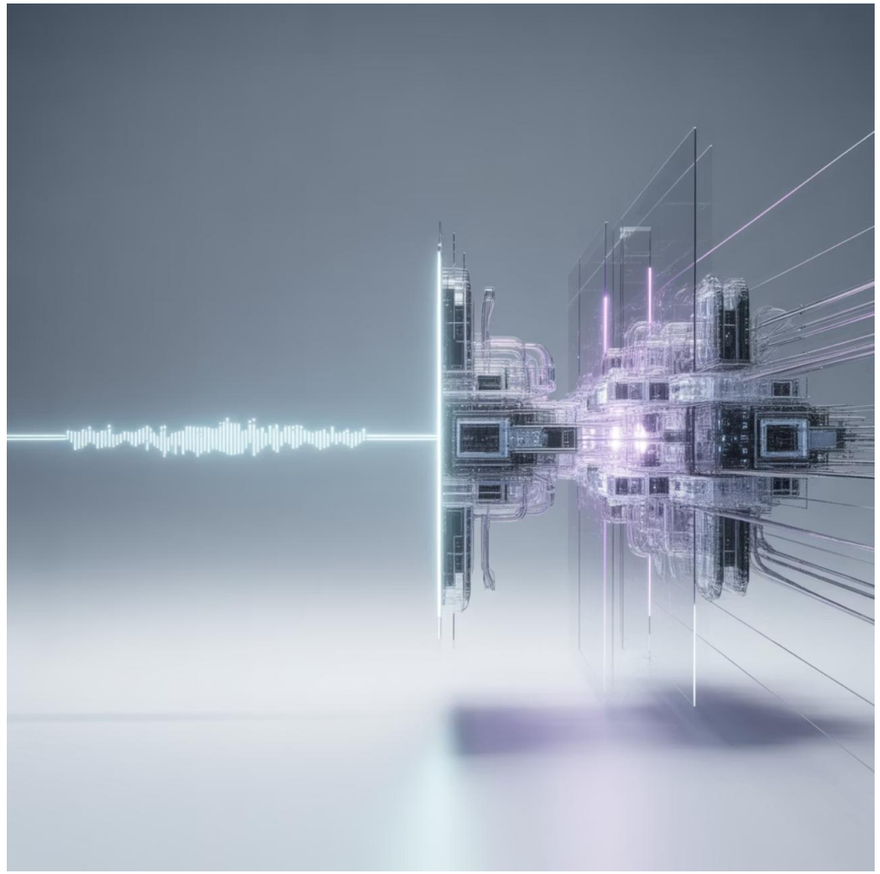
"Hello World"den Gerçek Projelere

Neden "Modern" Pratikler?

İlk başta yazdığımız küçük programlar tek bir dosyada olabilir. Birkaç satırlık bir kod parçasını yönetmek kolaydır ve her şey kontrolünüz altındadır.

Ancak projeler büyüdükçe – yüzlerce, hatta binlerce satır koda ulaştığında – organizasyon ve iş birliği hayati önem kazanır. Kod tabanınızın karmaşıklığı arttıkça, yapılandırılmış yaklaşımlara ihtiyaç duyarsınız.

Modern pratikler, kodun sürdürülebilir, anlaşılır ve ekip çalışmasına uygun olmasını sağlar. Profesyonel yazılım geliştirme, sadece çalışan kod yazmak değil, uzun vadede bakımı yapılabilir kod yazmaktır.





Fonksiyonlar: Tekrar Kullanılabilir Kod Blokları

Fonksiyon, belirli bir işi yapmak üzere tasarlanmış, isimlendirilmiş bir kod parçasıdır. Modern programlamanın temel yapı taşıdır.

"Don't Repeat Yourself"

Aynı kodu tekrar tekrar yazmayın – bir kez yazın, istediğiniz yerde kullanın

Kahve Makinesi Analojisi

Siz sadece düğmesine basarsınız (fonksiyonu çağırırsınız), o sizin için kahveyi yapar

Hazır Araç Kutusu

Her seferinde kahvenin nasıl yapıldığını baştan tarif etmenize gerek kalmaz

Bir Fonksiyon Nasıl Yazılır?

Her fonksiyonun belirli bir anatomisi vardır. Bu yapıyı anlamak, güçlü ve esnek kod yazmanın temelidir.

1

def Anahtar Kelimesi

Fonksiyonu tanımlamak için kullanılan başlangıç komutu

2

Fonksiyon Adı

Fonksiyonunuza verdiğiniz açıklayıcı ve anlamlı isim

3

Parametreler

Fonksiyonun çalışması için dışarıdan aldığı veriler (girdiler)

4

Fonksiyon Gövdesi

İşlemin gerçekleştiği kod bloğu

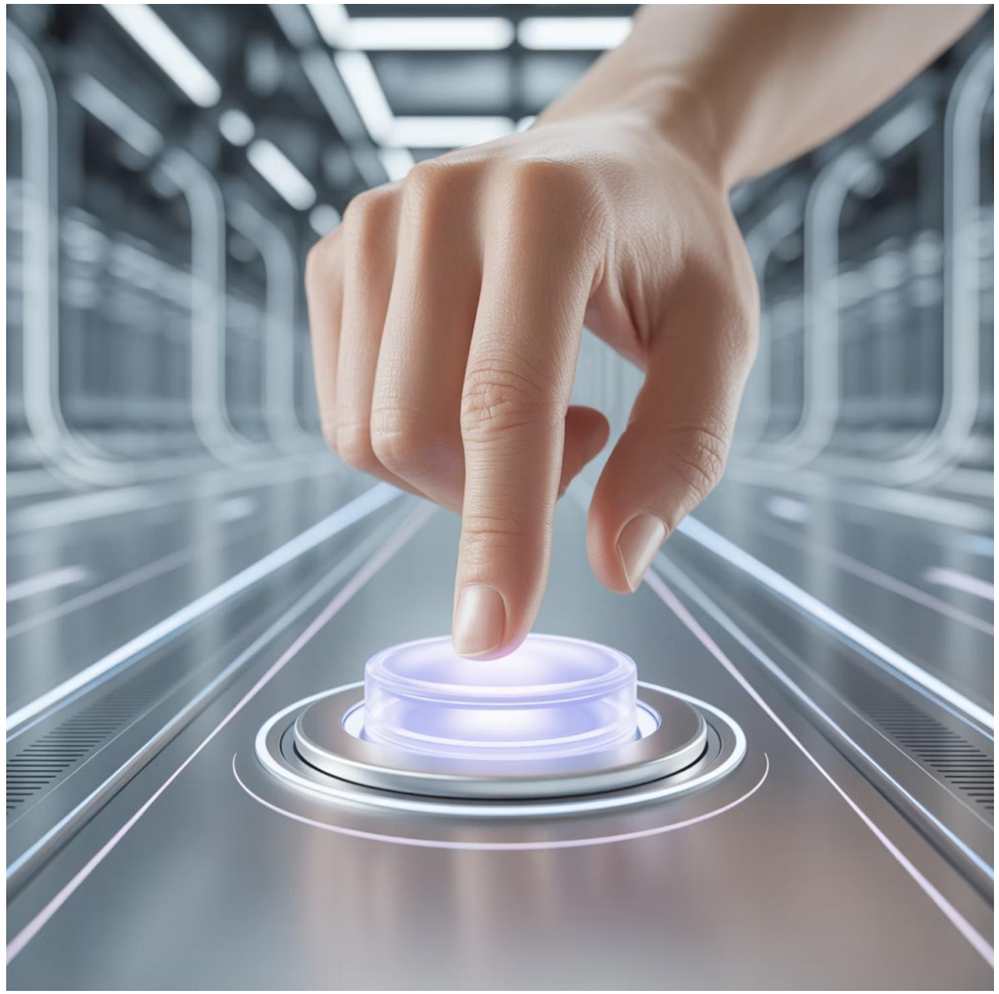
5

return İfadesi

Fonksiyonun yaptığı işlem sonucunda dışarıya döndürdüğü değer (çıktı)

```
def fonksiyon_adi(parametre1, parametre2): # Bu bloğun içindeki kodlar çalışır islem = parametre1 + parametre2 return islem # Fonksiyon bir değer döndürür
```

Kahve Makinesinin Düğmesine Basmak



Fonksiyonu Çağırma ve Kullanma

Fonksiyonu tanımladıktan sonra, onu programınızın istediğiniz yerinde çağırabilir ve kullanabilirsiniz. Bu, fonksiyonların asıl gücünü ortaya çıkarır.

```
# Fonksiyonu tanımlayalımdef topla(sayi1, sayi2):    return sayi1 + sayi2# Fonksiyonu çağıralımsonuc = topla(10, 25)print("İşlemin sonucu:", sonuc)# Ekrana sonuc)# Ekrana 35 yazar
```

Aynı fonksiyonu dilediğiniz kadar, farklı değerlerle çağırabilirsiniz. Her çağrıda yeni bir sonuç elde edersiniz.

Neden Her Yerde Fonksiyon Kullanmalıyız?

Fonksiyonlar, profesyonel yazılım geliştirmenin vazgeçilmez araçlarıdır. İşte size sağladığı temel avantajlar:



Tekrar Kullanılabilirlik

Reusability: Aynı kodu tekrar tekrar yazmaktan kurtarır. Bir kez yazın, her yerde kullanın. Kod tekrarı hem zaman kaybıdır hem de hata riskini artırır.



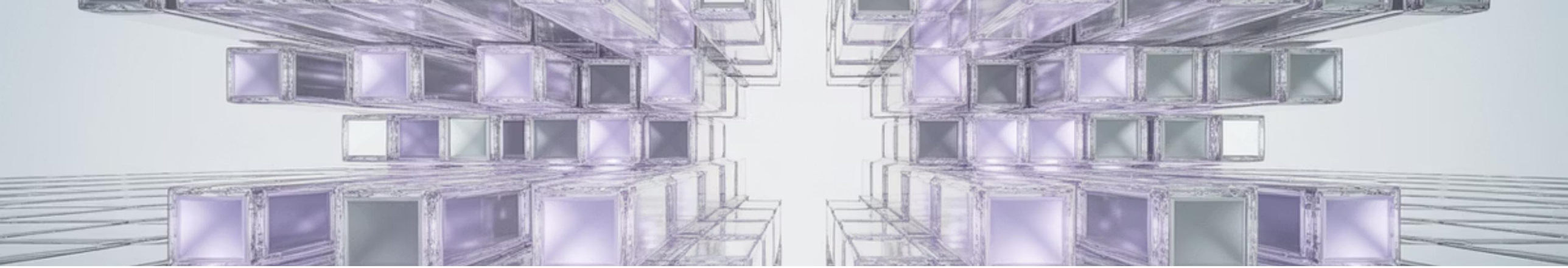
Okunabilirlik

Readability: Karmaşık bir problemi küçük, anlaşılır parçalara böler. `hesapla_kdv()` fonksiyonu, içinde ne yaptığını adıyla belli eder. Kod okuyan herkes ne yaptığını anlar.



Bakım Kolaylığı

Maintainability: Bir hesaplama yöntemini değiştirmeniz gerektiğinde, sadece ilgili fonksiyonu güncellemeniz yeterlidir. Tüm kodda değişiklik yapmaya gerek kalmaz.



Modüler Programlama Nedir?

Projeyi "Böl ve Yönet"

Modüler programlama, büyük bir programı, her biri belirli bir görevi yerine getiren daha küçük, bağımsız ve yönetilebilir parçalara (modüllere) ayırma tekniğidir.

🚗 Araba Analojisi

Bir araba düşünün: Motor, şanzıman, tekerlekler, elektronik sistem... Hepsi ayrı birer modüldür.

🔧 Kolay Bakım

Bir sorun olduğunda tüm arabayı değil, sadece ilgili modülü kontrol edersiniz. Modüler kod da böyle çalışır.

🧩 Bağımsız Parçalar

Her modül kendi işini yapar ve diğer modüllerden bağımsızdır. Bu sayede paralel geliştirme mümkün olur.

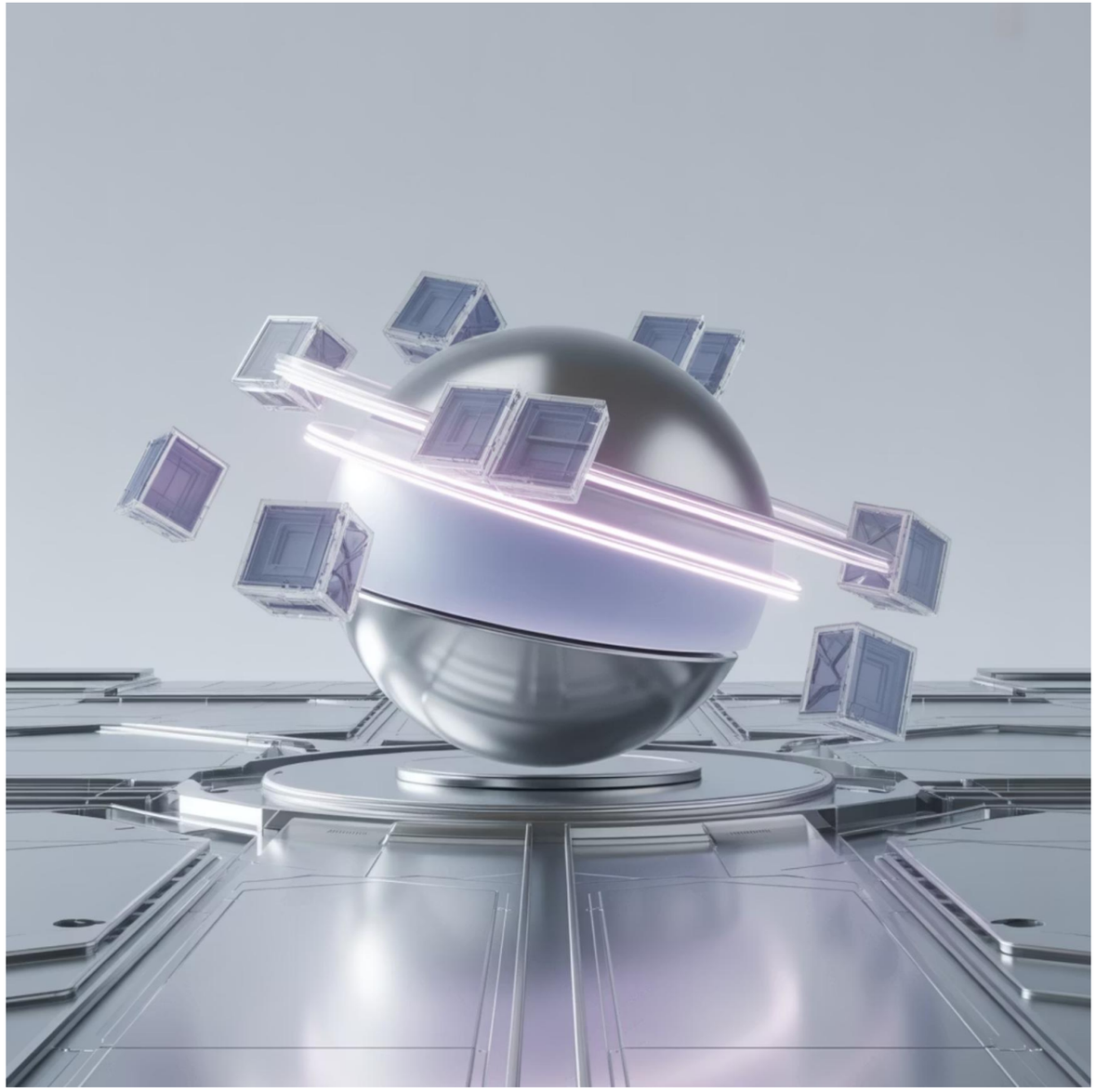
Python'da Modüller: import

Başkalarının Gücünü Kullanmak

Python'da her `.py` dosyası bir modüldür. `import` anahtar kelimesi ile başka dosyalardaki fonksiyonları veya değişkenleri kendi kodumuza dahil edebiliriz.

Python'un zengin standart kütüphanesi, hazır modüllerle doludur. Matematiksel işlemler, tarih/saat işlemleri, dosya okuma/yazma ve daha fazlası için hazır modüller mevcuttur.

```
import math# math modülü içindeki sqrt fonksiyonusayinin_karekoku = math.sqrt(16)print(sayinin_karekoku)  # 4.0# math
math.sqrt(16)print(sayinin_karekoku)  # 4.0# math modülü içindeki pi sabitiprint(math.pi)  # 3.141592...
```





Kendi Modülümüzü Yazmak

Kodumuzu Düzenleyelim

Kendi modüllerinizi oluşturmak, kodunuzu organize etmenin en güçlü yoludur. İşte basit bir örnek:

1

2

hesaplamalar.py

```
def topla(a, b):    return a + bdef carp(a, b):    return a * b
```

ana_program.py

```
import hesaplamalarsonuc1 = hesaplamalar.topla(5, 3)sonuc2 = hesaplamalar.carp(5, 3)print("Toplam:", sonuc1)  # 8print("Çarpım:", sonuc2)  # 15
```

Bu yaklaşımla, matematiksel işlemleriniz bir modülde, veri işleme fonksiyonlarınız başka bir modülde olabilir. Kod tabanınız büyüdükçe, bu organizasyon hayat kurtarıcı olur.



Kodunuz İçin Bir Zaman Makinesi

Versiyon Kontrol Sistemi (VCS) Nedir?

Versiyon kontrol sistemi, bir veya daha fazla dosya üzerinde yapılan değişiklikleri zaman içinde kaydeden ve bu sürümleri daha sonra geri çağırabilmenizi sağlayan bir sistemdir.

😱 Eski Yöntem

- proje_v1.py
- proje_v2.py
- proje_final.py
- proje_asıl_final_bu.py
- proje_bu_sefer_kesin_final.py

✨ Modern Yöntem

Tek bir dosya, sonsuz sürüm. Her değişiklik kaydedilir, istediğiniz zaman geri dönebilirsiniz. Kaos yok, sadece düzen var.

"CTRL+Z"nin Çok Ötesinde

Neden Versiyon Kontrolü Kullanmalıyız?

Versiyon kontrol sistemleri, modern yazılım geliştirmenin can damarıdır. İşte size sunduğu süper güçler:

📝 Değişiklik Takibi

Kim, neyi, ne zaman, neden değiştirdi? Her değişikliğin detaylı kaydını tutar. Kod tarihçeniz kristal berraklığında.

🔙 Hata Düzeltme

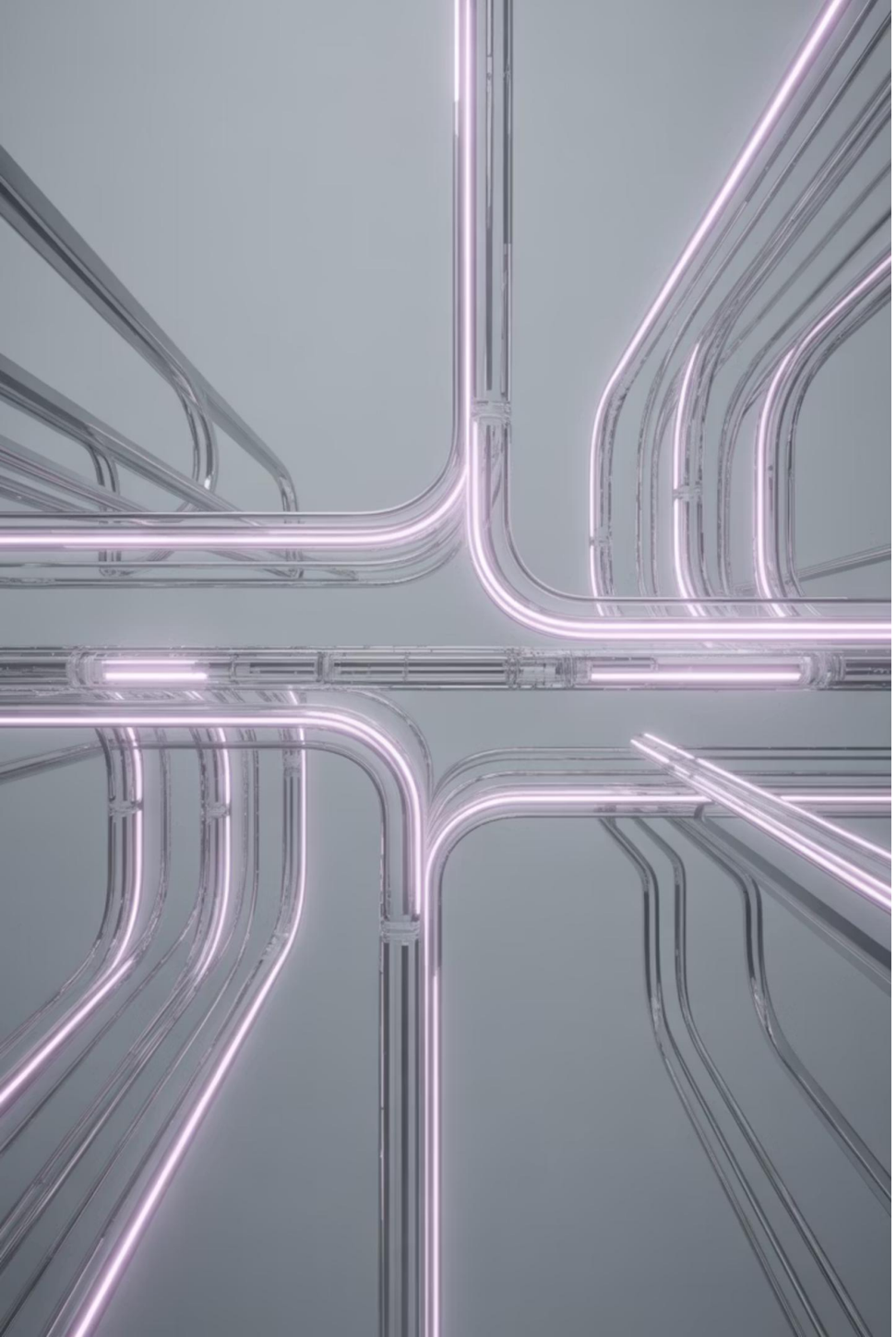
Çalışan bir kod parçasını bozdunuz mu? Panik yapmayın! Tek bir komutla çalışan eski sürüme dönebilirsiniz. Geriye dönük güvenlik ağınız her zaman hazır.

🌿 Paralel Geliştirme

Ana projeyi bozma riski olmadan yeni özellikler deneyebilirsiniz. Branching sayesinde güvenli bir kum havuzunda oynarsınız.

👥 Ekip Çalışması

Birden fazla kişinin aynı proje üzerinde sorunsuzca çalışmasını sağlar. Çatışmalar otomatik tespit edilir ve çözülür.



Tanıştıralım: Git ve GitHub

İkisi Aynı Şey Değil!

Bu iki kavram sıklıkla karıştırılır, ancak aralarında önemli farklar vardır. Her ikisini de anlamak, modern yazılım geliştirme için şarttır.

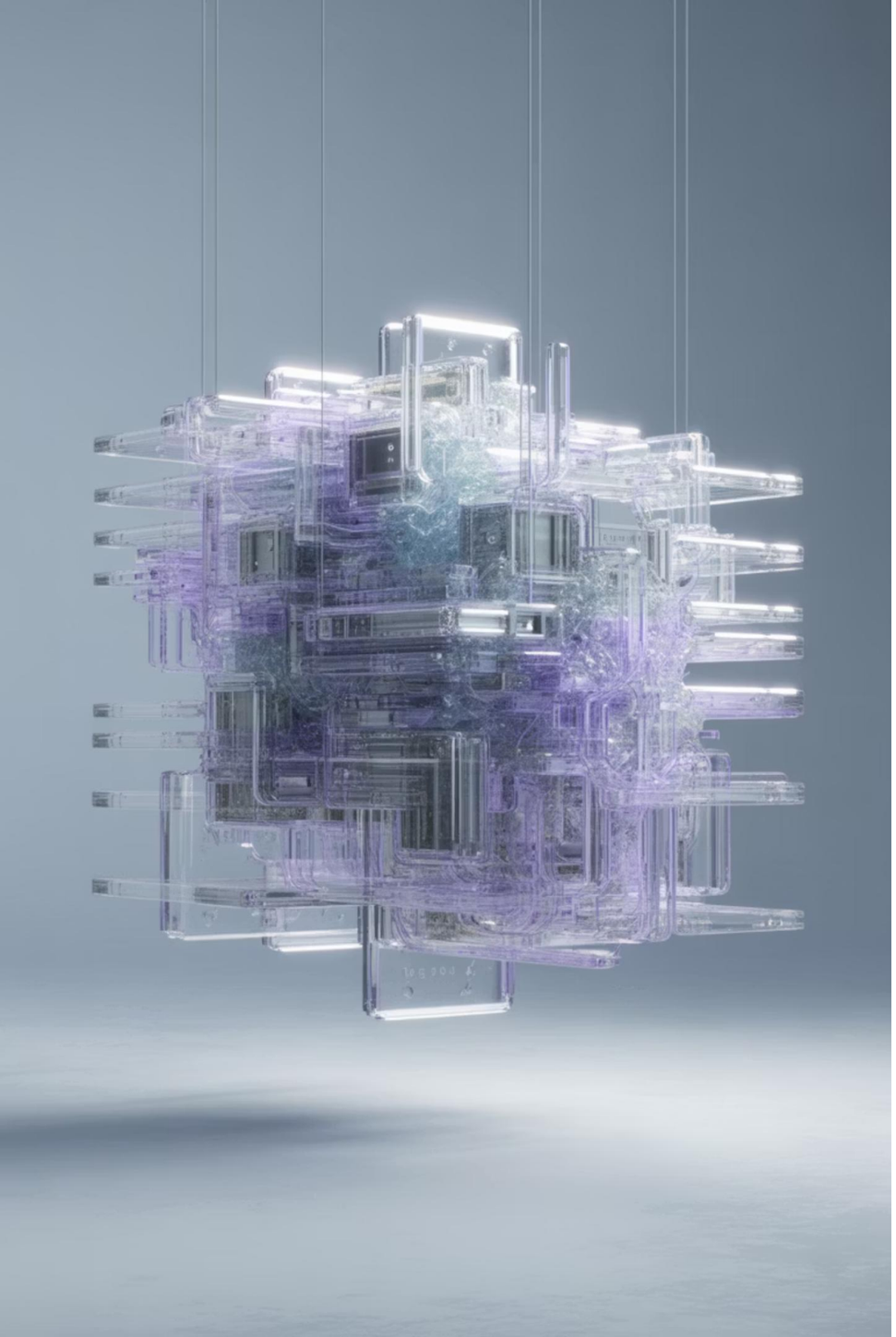
Git

Versiyon kontrolünü yapan **yazılımın kendisidir**. Bilgisayarınızda yerel olarak çalışır. Linus Torvalds tarafından 2005'te geliştirilmiştir.

GitHub

Git ile oluşturduğunuz kod depolarını **internet üzerinde** saklamanızı ve paylaşmanızı sağlayan bir web sitesidir (servistir).

🖼️ Analoji: Git, Microsoft Word programı gibidir; belgelerinizi oluşturur ve değişiklikleri takip edersiniz. GitHub ise Google Drive gibidir; Word belgelerinizi bulutta saklar, başkalarıyla paylaşırsınız ve birlikte düzenlersiniz.



Git'in Alfabesi

Temel Git Kavramları

Git'i etkili kullanabilmek için bu temel kavramları anlamanız gerekir. Bunlar, Git dünyasının yapı taşlarıdır.

Repository (Repo)

Projenizin tüm dosyalarını ve değişiklik geçmişini içeren klasör. Hem yerel bilgisayarınızda hem de GitHub'da bulunabilir.

Commit

Kodunuzun belirli bir andaki anlık görüntüsü, bir "kayıt noktası". Her commit'in açıklayıcı bir mesajı olmalıdır. İyi bir commit mesajı, gelecekteki kendinize armağandır.

Push

Yerel bilgisayarınızdaki (local) commit'leri GitHub'daki sunucuya (remote) gönderme işlemi. Değişikliklerinizi dünyayla paylaşırsınız.

Pull

GitHub'daki en son değişiklikleri kendi bilgisayarınıza çekme işlemi. Ekip arkadaşlarınızın yaptığı güncellemeleri alırsınız.

Günlük Kullanım Döngüsü

Basit Bir Git Akışı

Git kullanımı, tekrar eden basit bir döngüden oluşur. Bu döngüyü bir kez öğrendikten sonra, günlük iş akışınızın doğal bir parçası haline gelir.

1. Değişiklik Yap

Kodunu yaz veya düzenle. Dosyalarınızı dilediğiniz gibi değiştirin.



2. git add

Değişiklikleri "sahneye" ekle. Commit'e dahil edilecek dosyaları seç.

3. git commit

Sahnedeki değişiklikleri açıklayıcı bir mesajla kaydet.

4. git push

Kaydettiğin değişiklikleri GitHub'a yolla. Artık herkes görebilir.

```
# Örnek komutlargit add .git commit -m "Yeni hesaplama fonksiyonu eklendi"git push origin main
```

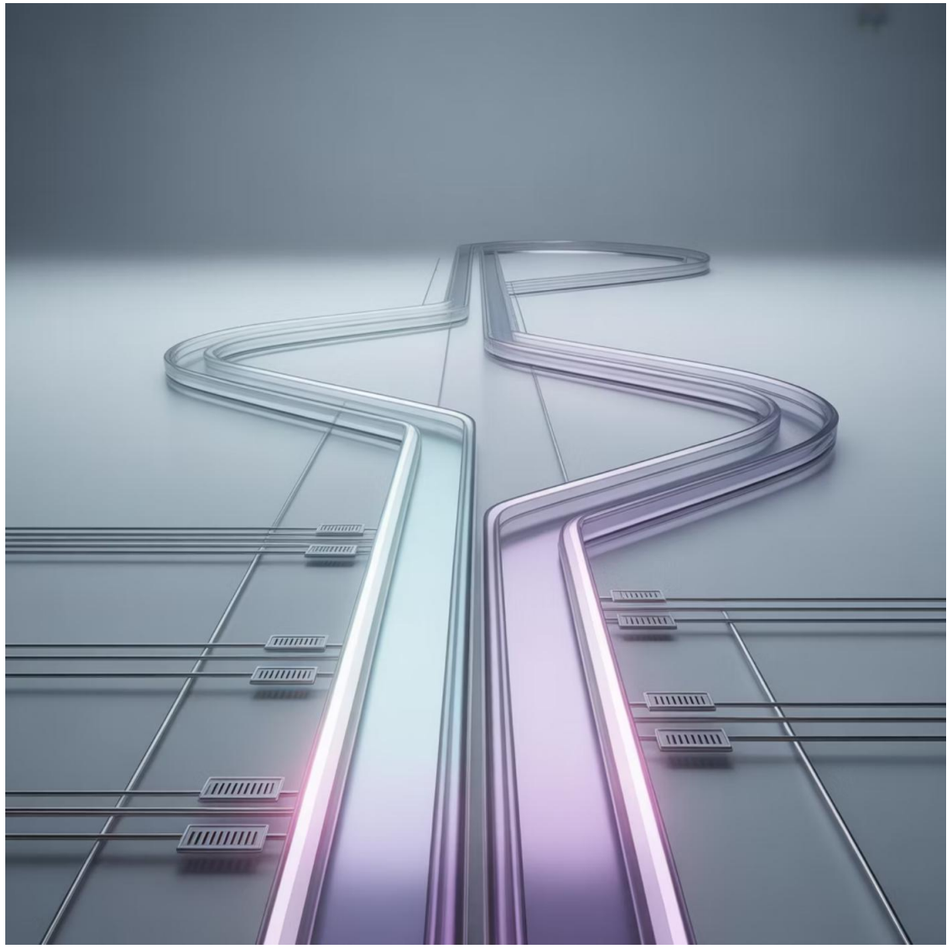
Projeyi Bozmadan Geliştirme

Branch (Dal) ve Merge (Birleştirme)

Branch, projenin ana hattını (main branch) etkilemeden yeni bir özellik veya deneme yapmak için oluşturulan paralel bir çalışma alanıdır.

Bir kitabın ana metnini bozmadan, kenarına notlar alarak yeni bir fikir geliştirmek gibidir. Branch'inizde özgürce deneyler yapabilir, hata yapabilirsiniz. Ana kod güvende kalır.

Merge ise ayrı bir branch'te tamamladığınız işi, projenin ana hattıyla birleştirme işlemidir. Yeni özelliğiniz test edildi ve hazır mı? Zamanı geldi, ana projeye entegre edin!



Ana Dal

Çalışan, stabil kod

1

2

Test & Geliştirme

Güvenli ortamda çalışma

3

4

GitHub Profili: Sizin Dijital CV'niz

Dünyaya Gösterdiğiniz Yüz

İş ve staj başvurularında, birçok şirket artık adayların GitHub profillerini inceliyor. Yaptığınız projeler, katkıda bulunduğunuz açık kaynaklı yazılımlar ve kodlama alışkanlıklarınız sizin en iyi referansınızdır.

Temiz ve Düzenli

Profilinizdeki projeler, README dosyalarıyla iyi dokümante edilmeli. İlk izlenim önemlidir.

Aktif Katkılar

Düzenli commit'ler, disiplininizi ve tutkunuzu gösterir. Yeşil kareler, çalışkanlığınızın kanıtıdır.

Çeşitli Projeler

Farklı teknolojilerle yapılmış projeler, öğrenme isteğinizi ve yeteneklerinizi sergiler.

💼 Kariyer İpucu: Temiz, düzenli ve aktif bir GitHub profili oluşturmak, kariyeriniz için yapacağınız en iyi yatırımlardan biridir. İşe alım yöneticileri, CV'nizden çok GitHub profilinize bakabilir!

Haftanın Aktivitesi / Ödevi - 1

Sıra Sizde!

Öğrenmenin en iyi yolu **pratik yapmaktır**. Aşağıdaki ödevleri yaparak bu hafta öğrendiğiniz kavramları pekiştireceksiniz.

1

Not Ortalaması Hesaplama

Kullanıcıdan vize ve final notlarını alın. Vize %40, final %60 etkili olacak şekilde ortalamasını hesaplayın. Eğer ortalama 50 veya üzerindeyse "Geçti", değilse "Kaldı" yazdırın.

Bonus: 85+ için "Takdir", 70-84 için "Teşekkür", 50-69 için "Geçti", altı için "Kaldı" mesajları ekleyin.

2

Çift Sayıları Yazdırma

1'den 100'e kadar olan çift sayıları ekrana yazdıran bir `for` döngüsü yazın.

İpucu 1: `range(2, 101, 2)` kullanabilirsiniz.

İpucu 2: Ya da `sayi % 2 == 0` kontrolü ile tek/çift ayırabilirsiniz.

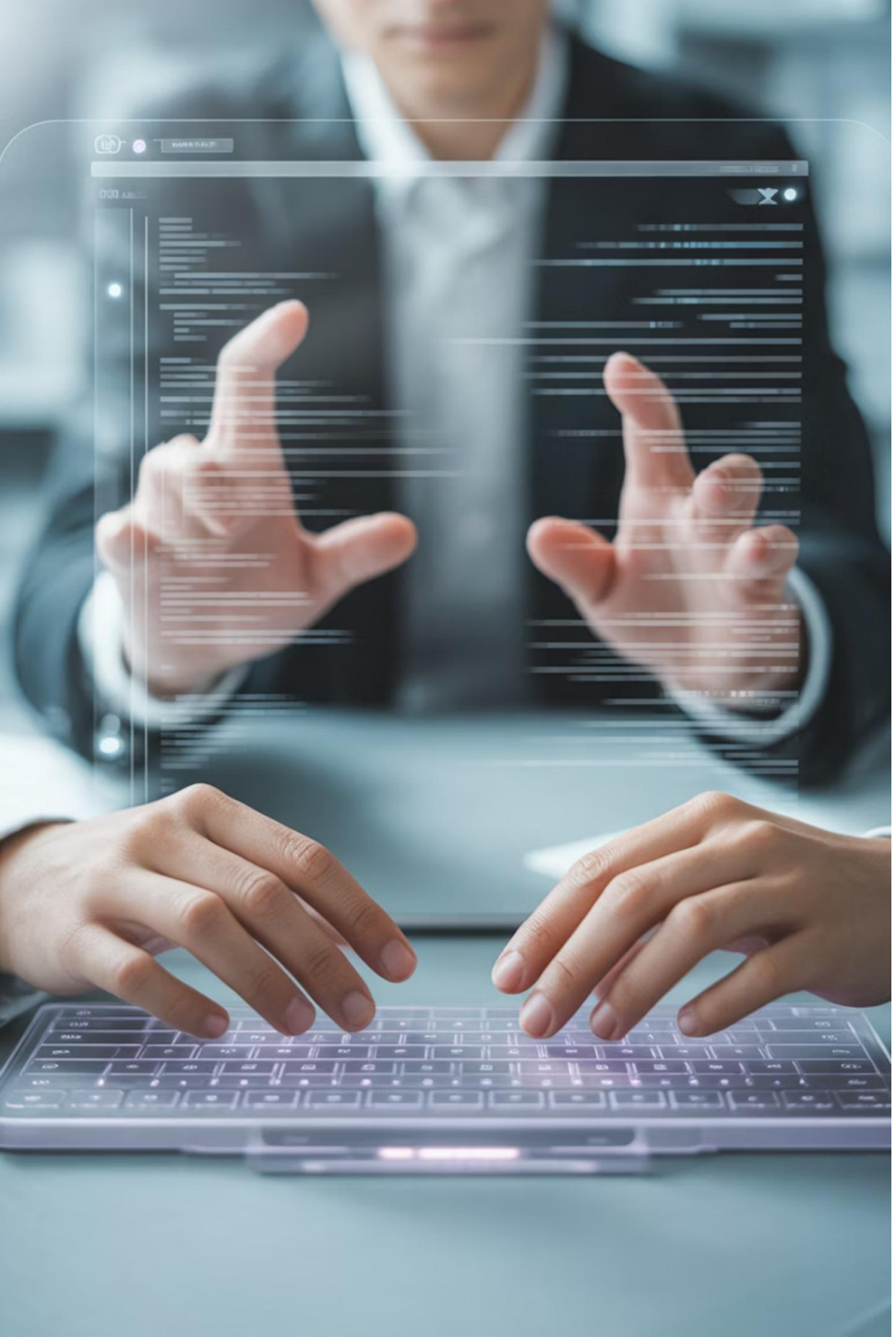
3

Bonus: Sayı Tahmin Oyunu

Slayt 19'daki sayı tahmin oyununu Python'da kodlayın. Kullanıcıya 5 tahmin hakkı verin ve her tahminden sonra "daha büyük" veya "daha küçük" ipucu gösterin.

Ekstra Bonus: Kullanıcı doğru bilirse kaç tahminde bulduğunu gösterin.

Teslim: Kodlarınızı bir `.py` dosyasına kaydedin ve yorumlarla açıklayın. Başarılar!



Kollarımızı Sıvayalım!

Haftanın Aktivitesi / Ödevi - 2

1

Bireysel Görev

- GitHub.com'a kaydolun ve profilinizi oluşturun
- Masaüstü için Git'i bilgisayarınıza kurun
- GitHub'da `bilgisayar-muh-giris` adında ilk repository'nizi oluşturun
- Bu repository'e içinde adınızın ve öğrenci numaranızın olduğu bir `README.md` dosyası ekleyip push edin

2

Ekip Çalışması

- 2-3 kişilik ekipler oluşturun
- Bir ekip üyesi bir repository oluştursun ve diğerlerini "collaborator" (işbirlikçi) olarak eklesin
- Ekip olarak, basit bir hesap makinesi programı yazın
- Her üye farklı bir fonksiyonu (toplama, çıkarma, çarpma, bölme) kendi branch'inde yazıp sonra `main` branch'e merge etsin

Bu aktivite, öğrendiğiniz tüm kavramları pratikte uygulama fırsatı sunar. Fonksiyonlar, modüler programlama, Git ve GitHub – hepsi bir arada!